Lecture 8 — Spark II: SQL & Machine Learning

CS4225 / CS5425 · Big Data Systems for Data Science · National University of Singapore

1 Spark SQL & DataFrames — What & Why

Core insight: Spark SQL is the layer that gives Spark the ability to understand *what* you want, not just *how* you said to get it. It exposes data as structured **DataFrames** — tables with named, typed columns — and an optimizer can then automatically improve your query before running it.

Spark SQL unifies all Spark components behind a single abstraction — the **DataFrame** (and its typed cousin, the **Dataset**). It is available in Java, Scala, Python, and R. Unlike raw RDDs, a DataFrame:

- Carries a **schema** — Spark knows the name and type of every column.
- Supports **optimized relational operations** — joins, aggregations, filters — using a query optimizer.
- Can be queried with **SQL strings** or with a **DSL** (domain-specific language) that looks like pandas.

Think of a DataFrame as a distributed SQL table. You can write `df.filter(df.age > 30).groupBy("city").count()` and Spark figures out the most efficient physical execution, just like a database planner would.

2 RDD vs. DataFrame — The Key Conceptual Difference

RDD = HOW | DataFrame = WHAT.
With an RDD you tell Spark exactly how to transform data step by step. With a DataFrame you declare what result you want, and Spark chooses how to get there.

Dimension	RDD	DataFrame / Dataset
Programming style	Imperative — you write each transformation explicitly	Declarative — you describe the desired result
Schema awareness	None — data is arbitrary Python/Scala objects	Full schema — column names and types known
Spark visibility	Opaque — user functions are black boxes; Spark cannot inspect or optimise them	Transparent — Spark can parse, inspect, and rewrite the query plan
Optimization	None — runs exactly as written	Catalyst Optimizer automatically rewrites for performance
Performance across languages	Python RDDs are slower (serialization overhead)	Same performance in Python, Scala, Java, R — optimizer bridges the gap
Ease of use	Lower — must think about partitions and transformations	Higher — DSL similar to pandas / SQL

Side-by-side code example — same task (average age per name) written both ways:

RDD approach

```python
# Create an RDD of (name, age) tuples
dataRDD = sc.parallelize([
    ("Brooke", 20), ("Denny", 31),
    ("Jules", 30), ("TD", 35),
    ("Brooke", 25)])

# HOW: map each record, reduceByKey to
# accumulate sum+count, then divide
agesRDD = (dataRDD
  .map(lambda x: (x[0], (x[1], 1)))
  .reduceByKey(lambda x, y:
      (x[0]+y[0], x[1]+y[1]))
  .map(lambda x:
      (x[0], x[1][0] / x[1][1])))
```

DataFrame approach

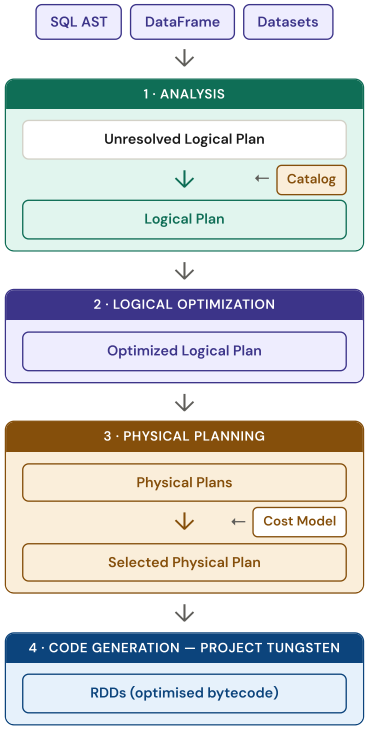
```python
# Create a DataFrame with a schema
data_df = spark.createDataFrame([
    ("Brooke", 20), ("Denny", 31),
    ("Jules", 30), ("TD", 35),
    ("Brooke", 25)],
    ["name", "age"])

# WHAT: group and aggregate — Spark
# decides the execution strategy
avg_df = data_df.groupBy("name").agg(avg("age"))
avg_df.show()

# +------+--------+
# | name |avg(age)|
# +------+--------+
# |Brooke|  22.5  |
# | Jules|  30.0  |
# |    TD|  35.0  |
# | Denny|  31.0  |
# +------+--------+
```

Notice that the RDD version manually encodes the averaging logic (map to (name, (age, 1)), reduceByKey to sum both, then divide). The DataFrame version just says `groupBy("name").agg(avg("age"))` — Spark figures out the rest. The DataFrame version is also shorter, easier to read, and automatically optimised.

Exam trap: A common question asks why Python DataFrames match Scala performance but Python RDDs do not. Answer: with DataFrames the Catalyst Optimizer generates the same JVM bytecode regardless of which language the query was written in. The Python layer is only used to *build* the query plan, not to execute it row by row.

3 The Catalyst Optimizer — Four-Phase Journey

Catalyst is the query optimizer that sits at the heart of Spark SQL. Whenever you write a DataFrame transformation or SQL query, Catalyst takes it through four phases before a single byte of data moves:



1. Analysis
Resolve column names and types against the schema. Build an Abstract Syntax Tree (unresolved → resolved logical plan).

→

2. Logical Optimisation
Apply rule-based rewrites to the logical plan: predicate pushdown, constant folding, column pruning, join reordering.

→

3. Physical Planning
Generate multiple physical plans (e.g. broadcast join vs sort-merge join). Pick the best using a cost model.

→

4. Code Generation
Project Tungsten compiles the chosen plan to optimised JVM bytecode — no interpreter overhead at runtime.

Predicate Pushdown (Phase 2)

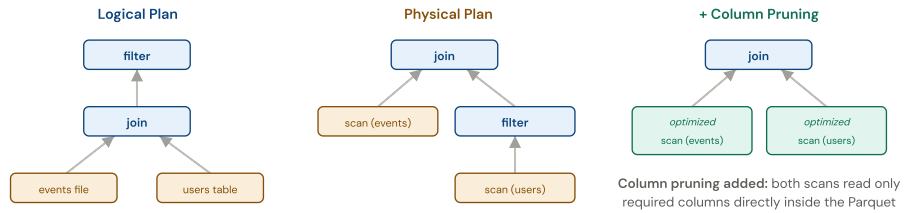
Filters are moved as early as possible — ideally into the data source — so fewer rows are ever read into memory. E.g. `WHERE age > 30` filters at scan time, not after a join.

Column Pruning (Phase 2)

Only columns actually referenced in the query are loaded. If a 100-column Parquet file is queried with `SELECT name, age`, only those two columns are deserialised.

Worked example — Catalyst optimising a join + filter query

```
// In Scala — join two DataFrames, then filter by date
val usersDF  = // Users DataFrame read from a Parquet table
val eventsDF = // Events DataFrame read from a Parquet table
val joinedDF = users
  .join(events, users("id") === events("uid"))
  .filter(events("date") > "2015-01-01")
```



As written: full join first, *then* filter — reads all rows from both tables.

Predicate pushdown: filter moved before the join — only matching users rows enter the join.

Column pruning added: both scans read only required columns directly inside the Parquet file reader.

> The code `.join().filter()` is written with the filter last, but Catalyst rewrites the execution so the filter runs before the join and unnecessary columns are never loaded from disk. The final *result* is identical; the *cost* is dramatically lower — especially important at scale.

4 Project Tungsten — Hardware-Level Optimisation

Project Tungsten is Spark's initiative to push performance as close as possible to the theoretical limits of modern hardware (CPU and memory). It operates at the code-generation phase of Catalyst.

Memory Management & Binary Processing

Manages memory **off-heap** in a compact binary format, bypassing Java object overhead and garbage collection (GC) pauses. Avoids GC pressure that slows JVM programs.

Cache-Aware Computation

Algorithms and data structures are arranged to maximise CPU cache hits. Sorting and aggregation operate on binary data laid out to fit CPU cache lines.

Whole-Stage Code Generation

Instead of interpreting operators row-by-row, Tungsten generates a single tight loop of JVM bytecode for an entire pipeline stage — eliminating virtual function dispatch overhead.

> The combined effect: **language performance parity**. A Python DataFrame query produces the same generated JVM bytecode as an equivalent Scala query — so Python users get Scala-level performance when using the DataFrame/SQL API.

5 Machine Learning with Spark ML — Overview

Core insight: Spark ML (part of MLlib) brings the full supervised machine learning workflow — preprocessing, training, evaluation — into the distributed Spark framework, so it scales to datasets that don't fit on one machine.

Supervised ML tasks come in two flavours:

Classification

Predict a **categorical label** (a class) from features.
Example: given an email's words, predict *spam* or *not spam*.
Inputs: feature matrix X_{train}, label vector y_{train} (discrete classes).

Regression

Predict a **numeric value** from features.
Example: given patient vitals, predict *length of hospital stay* in days.
Inputs: feature matrix X_{train}, label vector y_{train} (continuous numbers).

Example 1 — Classification: Spam Detection

Step 1 — Training: the model sees labelled rows and learns which feature patterns correspond to each class.

Input features X_{train}				Label y_{train}
word_count	has_link	from_unknown	exclamations	
45	1 (yes)	1 (yes)	3	1 — spam
120	0 (no)	0 (no)	0	0 — not spam
30	1 (yes)	1 (yes)	5	1 — spam
200	0 (no)	0 (no)	1	0 — not spam

Step 2 — Prediction: given an unseen email, the model outputs a predicted class.

Input features X_{test}				Prediction $\hat{y}_{test}$
word_count	has_link	from_unknown	exclamations	
25	1 (yes)	1 (yes)	4	1 — spam

Output is a **discrete class label**. The model fires on the combination of has_link + from_unknown + high exclamation count, all of which were associated with spam in the training data.

Example 2 — Regression: Hospital Stay Prediction

Step 1 — Training: the model sees labelled rows and learns the relationship between patient features and length of stay.

Input features X_{train}				Label y_{train}
age	blood_pressure	diagnosis	num_meds	
65	140	cardiac	5	12 days
45	120	diabetes	2	3 days
72	160	cardiac	8	18 days
30	115	minor	1	1 day

Step 2 — Prediction: given an unseen patient, the model outputs a predicted numeric value.

Input features X_{test}				Prediction $\hat{y}_{test}$
age	blood_pressure	diagnosis	num_meds	
58	135	cardiac	6	9.4 days

Output is a **continuous number** — not a class. 9.4 days lies between the 3-day diabetes patient and the 12-day cardiac patient, which makes intuitive sense given the features.

Key difference: classification outputs a category (spam / not spam); regression outputs any real number (9.4 days). This determines which loss function and evaluation metrics you use.

The typical ML pipeline has four phases:

Preprocessing
Process raw data — handle missing values, encode categoricals, normalise features.

→

Training
Fit a model to (X_{train}, y_{train}) — learn parameters that minimise a loss function.

→

Testing
Apply the trained model to unseen X_{test} to produce predictions $\hat{y}_{test}$.

→

Evaluation
Compare $\hat{y}_{test}$ against ground truth y_{test} using accuracy, F1, RMSE, etc.

6 Data Preprocessing — Cleaning & Feature Engineering

> "Garbage in, garbage out." Google AI (SIGCHI 2021) found that 92% of high-stakes AI projects reported downstream problems caused by data quality issues. NYT reported that data scientists spend 50–80% of their time on data preparation.

Missing Values

Data can be missing because it was never collected, or because it is *missing not at random* (in which case the missingness is itself informative). Options:

1. **Drop rows** — eliminate any record that has a missing value. Simple but wasteful if many rows are affected.
2. **Mean/median imputation** — replace each missing value with the column's mean or median. Preserves row count. `Imputer` in PySpark handles this.
3. **Regression imputation** — build a secondary model to predict the missing column from other columns. More accurate but more complex.
4. **Dummy variable** — after imputing, optionally add an indicator column (1 = was missing, 0 = was present) so the model can learn whether missingness itself is a signal.

```
from pyspark.ml.feature import Imputer
imputer = Imputer(inputCols=["height", "weight"], outputCols=["height_out", "weight_out"])
model = imputer.fit(df)
model.transform(df).show()
```

Categorical Encoding

Ordinal Encoding

Map categories to integers that preserve an inherent order.

Example: Risk = {Low, Medium, High} → {0, 1, 2}.

Use when there IS a natural ordering among categories (e.g. severity levels). Algorithms like linear regression can exploit the ordering.

One-Hot Encoding (OHE)

Convert a categorical column with K values into K binary columns; exactly one column is 1 per row.

Use when there is NO ordinal relationship. Prevents the model from inferring a false numeric ordering.

OHE worked example — Group column with 3 categories {A, B, C} expands into 3 binary columns:

index	group
0	B
1	A
2	C

→

index	GroupA	GroupB	GroupC
0	0	1	0
1	1	0	0
2	0	0	1

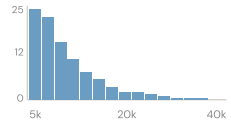
> **Exam trap:** Applying ordinal encoding to a non-ordinal feature (e.g. colours: red=0, blue=1, green=2) implies blue is "between" red and green, which is meaningless and can hurt model performance. Use one-hot encoding instead.

Normalisation / Scaling

Many algorithms (especially gradient-based ones like logistic regression) are sensitive to the scale of features. Normalisation brings all features onto a comparable range.

Each histogram below shows the distribution of a *Price* feature after applying each method. The raw feature is right-skewed (most prices are low, with a long tail of high prices). Notice how only the x-axis scale changes for linear transforms, while clipping and log actively reshape the distribution.

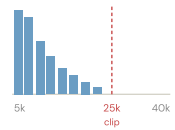
Price (raw feature)



→

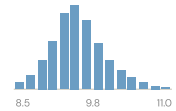
> The raw Price feature is **right-skewed** — the tallest bars are at the low end (cheap properties are common) and the distribution has a long tail to the right (a few very expensive properties). This skew can hurt ML models that assume roughly equal feature ranges. The four methods below each handle it differently.

Clipping — clip(x)



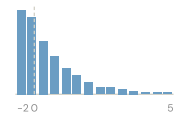
Cap values at a min and max threshold. Removes extreme outliers without changing the rest.

Log Transform — log(1+x)



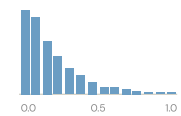
Apply log(1 + x). Compresses large-valued, right-skewed features (e.g. income, counts) into a more Gaussian-like distribution.

Standard Scaler



Subtract mean, divide by std: (x - mean) / std. Output has mean 0 and std 1.

Max Min Normalisation



Scale to [0, 1]: (x - min) / (max - min). Useful when the exact range matters.

7 Logistic Regression — Algorithm Deep Dive

Logistic regression is the canonical binary classification algorithm. Despite the name "regression", it predicts a *probability* which is then thresholded to give a class label.

The Sigmoid Function

The sigmoid (logistic) function squashes any real number into the range (0, 1), making it interpretable as a probability:

```
σ(z) = 1 / (1 + e⁻ᶻ)

where  z = w · x + b   (dot product of weights and input features, plus bias)

σ(z) → 1  as  z → +∞   (model is very confident of class 1)
σ(z) → 0  as  z → -∞   (model is very confident of class 0)
σ(0) = 0.5            (model is uncertain)
```

Parameters: The model learns weights vector `w` (one weight per feature) and a bias term `b`. The prediction for a new sample x is:

```
ŷ = σ(w · x + b)    # predicted probability of class 1
predicted class = 1 if ŷ ≥ 0.5, else 0
```

Loss Function — Cross-Entropy Loss

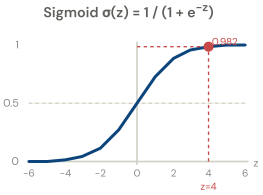
To train the model, we define a loss that measures how wrong the predictions are. Cross-entropy loss is low when predicted probability is close to the true label, and high when it is far:

```
J(w, b) = -(1/n) Σᵢ [ yᵢ · log(σ(wᵀxᵢ + b)) + (1 - yᵢ) · log(1 - σ(wᵀxᵢ + b)) ]

For a positive example (y=1): penalises predictions close to 0  (high loss if ŷ ≈ 0)
For a negative example (y=0): penalises predictions close to 1  (high loss if ŷ ≈ 1)
```

The big picture: **machine learning = fitting parameters by minimising a loss function**. Here we fit w and b to minimise the cross-entropy loss J(w, b) over all training samples.

Worked example — Spam Classification (from slides)

An email has 5 dollar signs and 2 exclamation marks. Given learned weights **w = (1, 2)** and bias **b = −5**, what is the predicted spam probability?



1. **Set up the input vector**
   Features: number of '$' = 5, number of '!' = 2
   `x = (5, 2)` `w = (1, 2)` `b = -5`
2. **Compute the linear combination z = w · x + b**
   `z = (1×5) + (2×2) + (-5)`
   `z = 5 + 4 - 5 = 4`
3. **Apply the sigmoid function**
   `σ(z) = 1 / (1 + e⁻⁴)`
   `e⁻⁴ ≈ 0.0183`
   `σ(4) = 1 / 1.0183 ≈ 0.982`
4. **Threshold to get the predicted class**
   `ŷ = 0.982 ≥ 0.5` → predicted class = **1 (spam)**
   The model is 98.2% confident this email is spam.

Note: z = 4 is far to the right of 0 on the sigmoid curve — the output is very close to 1. If z were 0 (e.g. the features exactly cancelled the bias), ŷ = 0.5 and the model would be maximally uncertain.

8 Gradient Descent & Parallel Gradient Descent in Spark

Gradient Descent — Intuition

The loss surface J(w, b) is a landscape of hills and valleys. Gradient descent starts at a random point on that landscape and iteratively takes small steps in the direction of steepest descent (opposite to the gradient) until it reaches a minimum.

1. Initialise weights w randomly (or to zero).
2. Compute the gradient of J with respect to w — this tells us which direction increases the loss.
3. Move w in the *opposite* direction by a small amount controlled by the **learning rate** α (step size).
4. Repeat until the improvement in J falls below a threshold (convergence).

```
# Update rule (vectorised over all n training samples with d features)
# w is a d-dimensional weight vector

w^(i+1)  ←  w^(i)  -  α · (1/n) · Xᵀ · (σ(Xw) - y)

# α       = learning rate (step size)
# X       = (n × d) feature matrix
# y       = (n × 1) true label vector
# σ(Xw)   = (n × 1) vector of predicted probabilities
# Xᵀ · (σ(Xw) - y)  = gradient (direction of steepest ascent in loss)
```

Exam trap: Gradient descent is *iterative* — it requires multiple passes over the full training set. This is exactly what makes it expensive on big data and why we need Spark to parallelize it.

Parallel Gradient Descent in Spark

The key insight: the gradient is a *sum* over all training examples. Sums are trivially parallelisable — split the data, compute partial sums on each node, then aggregate.

Broadcast Weights
Driver broadcasts current weight vector w to all worker nodes. w is a small vector (d floats) — cheap to send.

→

Local Gradient
Each worker computes the gradient on its own partition of the training data using the received w. No inter-worker communication needed here.

→

Aggregate
All partial gradients are summed back at the driver (reduce step). Driver averages and updates w.

→

Repeat
Loop until convergence. Training data is cached in memory — each iteration reads from the cache, not disk.

Broadcast Variable

The weight vector w is a **broadcast variable** — Spark sends it efficiently to all nodes at the start of each iteration, and workers read it from their local cache rather than fetching it from the driver per record.

Cached Training Data

Training data is `.cache()`-d so it lives in memory across all gradient descent iterations. Without caching, Spark would re-read data from disk on every iteration — catastrophically slow.

Why this scales: The training data (large) is partitioned and stays on workers. Only the weight vector (tiny — d numbers) crosses the network each iteration. Communication cost is O(d), not O(n).

Worked example — n = 6 samples, 3 workers (from slide 44)

Continuing the spam example. Features: x_1 = number of '$', x_2 = number of '!'. Current weights: **w = (0, 0)** (untrained). Learning rate: **α = 0.1**. Since w = (0,0), σ(x · w) = σ(0) = 0.5 for every sample.

Step 1 — Driver broadcasts w = (0, 0) to all workers. Training data is cached in partitions:

Worker 1 — samples j=1, j=5

j	x_1	x_2	y
1	5	2	1
5	3	1	1

Worker 2 — samples j=3, j=4

j	x_1	x_2	y
3	4	3	1
4	0	1	0

Worker 3 — samples j=2, j=6

j	x_1	x_2	y
2	1	0	0
6	2	0	0

Step 2 — Map: each worker independently computes its partial gradient using $[\sigma(x^{(j)} \cdot w) - y^{(j)}] \times x^{(j)}$

Since w = (0,0), σ(0) = 0.5 for all samples, so the error term is simply (0.5 - y).

Worker 1 local gradient

j=1: (0.5−1)×(5,2) = **(−2.5, −1.0)**
j=5: (0.5−1)×(3,1) = **(−1.5, −0.5)**

Subtotal: **(−4.0, −1.5)**

Worker 2 local gradient

j=3: (0.5−1)×(4,3) = **(−2.0, −1.5)**
j=4: (0.5−0)×(0,1) = **(0.0, +0.5)**

Subtotal: **(−2.0, −1.0)**

Worker 3 local gradient

j=2: (0.5−0)×(1,0) = **(+0.5, 0.0)**
j=6: (0.5−0)×(2,0) = **(+1.0, 0.0)**

Subtotal: **(+1.5, 0.0)**

Step 3 — Reduce: driver sums all partial gradients, then updates w

Reduce — aggregate partial gradients

(−4.0, −1.5) [Worker 1]
+ (−2.0, −1.0) [Worker 2]
+ (+1.5, 0.0) [Worker 3]

Total gradient = (−4.5, −2.5)

Update — new weights

$w_{new} = w_{old} - \alpha \times \text{gradient}$
$w_{new} = (0, 0) - 0.1 \times (-4.5, -2.5)$

w_{new} **= (+0.45, +0.25)**

After just one iteration both weights became **positive** — the model has started to learn that more '$' signs and '!' marks increase the spam probability. In practice this loop repeats hundreds of times until w converges.

9 Model Evaluation Metrics

Classification — Confusion Matrix

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Green = correct | Red = wrong

TP — True Positive: predicted Positive, actually Positive. Correct.

TN — True Negative: predicted Negative, actually Negative. Correct.

FP — False Positive: predicted Positive, actually Negative. "False alarm."

FN — False Negative: predicted Negative, actually Positive. "Missed detection."

> In spam detection: FP = good email wrongly flagged as spam. FN = spam that slipped through. Both are errors but with different costs.

```
Accuracy  = (TP + TN) / (TP + FP + FN + TN)
            # fraction of all predictions that are correct

Precision = TP / (TP + FP)
            # of all predicted positives, how many are actually positive?

Recall    = TP / (TP + FN)
            # of all actual positives, how many did we catch?

F1 Score  = 2 × (Precision × Recall) / (Precision + Recall)
            # harmonic mean of precision and recall; balances precision and recall
```

> **Precision vs Recall trade-off:** In spam detection, high precision = rarely flag good email as spam (few false alarms). High recall = rarely miss actual spam. Lowering the classification threshold increases recall but lowers precision. F1 balances both.

Regression Evaluation

MAE — Mean Absolute Error

$$MAE = \frac{1}{n} \sum_i |\hat{y}_i - y_i|$$

Average of $|\hat{y}_i - y_i|$. Lower is better. Easy to interpret in original units. Less sensitive to outliers.

MSE / RMSE — Mean Squared Error

$$MSE = \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$$

Average of $(\hat{y}_i - y_i)^2$. Lower is better. Penalises large errors heavily. RMSE = $\sqrt{MSE}$ — back in original units.

R^2 — Coefficient of Determination

$$R^2 = 1 - SS_{res} / SS_{tot}$$

$SS_{res} = \sum(y_i - \hat{y}_i)^2$ — error from your model

$SS_{tot} = \sum(y_i - \bar{y})^2$ — error from always predicting the mean

Higher is better. Measures what fraction of the spread in y your model explains. The baseline is "always predict the mean $\bar{y}$" — R^2 compares your model against that.

R^2=1 — perfect fit. **R^2=0** — no better than predicting the mean. **R^2<0** — worse than predicting the mean ($SS_{res} > SS_{tot}$), usually signals a broken model.

Why R^2 can go negative — worked example

Baseline — always predict the mean $\bar{y}$ = 3

Actual y = [1, 2, 3, 4, 5], $\bar{y}$ = 3

Errors = $|1-3|^2, |2-3|^2, |3-3|^2, |4-3|^2, |5-3|^2$

= 4 + 1 + 0 + 1 + 4

SS_{tot} = 10

Terrible model predictions = [5, 1, 5, 1, 5]

Errors = $|1-5|^2, |2-1|^2, |3-5|^2, |4-1|^2, |5-5|^2$

= 16 + 1 + 4 + 9 + 0

SS_{res} = 30

R^2 = 1 - (30 / 10) = 1 - 3 = **-2**

> The model's predictions are so wrong that you would genuinely get a better result by ignoring all features and just guessing the mean every time. In practice $R^2 < 0$ almost always means something went wrong — mismatched train/test data, features with no relationship to y, or a pipeline bug.

10 Spark ML Pipelines — Transformers & Estimators

Core insight: A **Pipeline** chains simple building blocks (Transformers and Estimators) into a reproducible, reusable ML workflow. The same pipeline object handles both training and inference — keeping preprocessing and model steps in sync automatically.

Transformer

An object that maps a DataFrame to another DataFrame — it *transforms* data.

Method: `transform(df) → df'`

Examples: `OneHotEncoder`, `StandardScaler`, `Tokenizer`.

A *fitted model* (e.g. a trained LogisticRegression) is also a Transformer — it appends a predictions column.

Estimator

An object that takes data and *learns* from it, producing a fitted Transformer.

Method: `fit(df) → Transformer`

Examples: `LogisticRegression`, `StandardScaler` (fit learns mean & std), `Imputer` (fit learns medians).

Note: many preprocessing steps are Estimators at fit time, Transformers at transform time.

Pipeline at Training Time

A `Pipeline` is itself an Estimator. When you call `Pipeline.fit(training_data)`, it walks through each stage in order:

1. If the current stage is a **Transformer**, call `stage.transform(data)` and pass the result to the next stage.
2. If the current stage is an **Estimator**, call `stage.fit(data)` to get a fitted model (a Transformer), then immediately call `fitted_model.transform(data)` and pass that result to the next stage.
3. `Pipeline.fit()` returns a **PipelineModel** — a sequence of the fitted Transformers produced in step 2.

Pipeline at Test / Inference Time

A `PipelineModel` is a Transformer. Calling `pipeline_model.transform(test_data)` runs each fitted Transformer's `transform()` in order — applying exactly the same preprocessing and model that was fit during training.

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
from pyspark.ml.classification import LogisticRegression

# Define stages
indexer = StringIndexer(inputCol="category", outputCol="cat_index")
encoder = OneHotEncoder(inputCol="cat_index", outputCol="cat_ohe")
assembler = VectorAssembler(inputCols=["cat_ohe", "age", "income"], outputCol="features")
lr = LogisticRegression(maxIter=10, labelCol="label")

# Chain into a pipeline
pipeline = Pipeline(stages=[indexer, encoder, assembler, lr])

# Training: fit on training data → PipelineModel
pipeline_model = pipeline.fit(train_df)

# Inference: transform test data (applies same pipeline)
predictions = pipeline_model.transform(test_df)
```

Why use Pipelines?

✓ **Code reuse**

Without pipelines, you must manually apply the same preprocessing steps during training, validation, and test — and keep them in sync by hand. A pipeline does this automatically.

✓ **Cross-validation & hyperparameter tuning**

Spark's `CrossValidator` and `TrainValidationSplit` expect an Estimator — passing a full Pipeline means preprocessing is refitted correctly on each fold, preventing data leakage.

Data leakage warning: If you fit your scaler or imputer on the full dataset before splitting into train/test, statistics from the test set "leak" into the training process. Always fit preprocessing inside the pipeline so it only sees training data.

11 Comparison Tables

Transformer vs Estimator

Dimension	Transformer	Estimator
Purpose	Transforms one DataFrame into another	Learns from data to produce a Transformer
Key method	`transform(df) → df'`	`fit(df) → Transformer`
Examples	OneHotEncoder, Tokenizer, fitted model	LogisticRegression, StandardScaler, Imputer
Has learnable params?	No (already fixed)	Yes (learns params during `fit()`)
In a Pipeline	transform() called directly	fit() called first, then transform()

Classification vs Regression Evaluation

Metric	Task	Formula	Good value
Accuracy	Classification	(TP+TN)/(TP+FP+FN+TN)	Higher
Precision	Classification	TP / (TP+FP)	Higher
Recall	Classification	TP / (TP+FN)	Higher
F1 Score	Classification	2·(P·R)/(P+R)	Higher
MAE	Regression	mean(|ŷ−y|)	Lower
RMSE	Regression	√mean((ŷ−y)²)	Lower
R^2	Regression	$1 - SS_{res}/SS_{tot}$	Higher (→1)

12 Exam-Style Q&A

Q: Why do Python and Scala DataFrames have the same performance, while Python RDDs are slower than Scala RDDs?

> With DataFrames, Spark SQL's Catalyst Optimizer generates the same JVM bytecode (via Project Tungsten's whole-stage code generation) regardless of the language used to write the query. Python is only used to construct the logical plan — execution happens entirely in the JVM. With RDDs, Python user functions must be executed by a Python process (via PySpark serialization), incurring serialization/deserialization overhead per record.

Q: What are the four phases of the Catalyst Optimizer? Briefly describe each.

> (1) **Analysis** — resolve column names and types against the schema to produce a resolved logical plan. (2) **Logical optimisation** — apply rule-based transformations such as predicate pushdown, constant folding, and column pruning. (3) **Physical planning** — enumerate multiple physical execution strategies (e.g. broadcast join vs sort-merge join) and select the cheapest using a cost model. (4) **Code generation** (Project Tungsten) — compile the chosen physical plan into optimised JVM bytecode for execution.

Q: How does Spark parallelise gradient descent for logistic regression?

> The weight vector w is broadcast to all worker nodes (it is a small d-dimensional vector). The training data is partitioned across workers and cached in memory. Each worker computes the gradient of the loss on its own partition. The partial gradients are aggregated (summed) back to the driver, which averages them, multiplies by the learning rate α, and subtracts from w to get the updated weights. This loop repeats until convergence. The key efficiency: only w (d numbers) crosses the network each iteration, not the full training set.

Q: Distinguish Precision and Recall. When would you prioritise each?

> **Precision** = TP/(TP+FP) — of everything the model called positive, how many actually were? High precision means few false alarms. **Recall** = TP/(TP+FN) — of all actual positives, how many did the model find? High recall means few misses. Prioritise *precision* when false positives are costly (e.g. spam filter — you don't want to lose legitimate emails). Prioritise *recall* when false negatives are costly (e.g. cancer screening — you don't want to miss a real case).

Q: What is the difference between a Transformer and an Estimator in Spark ML?

> A **Transformer** maps a DataFrame to a new DataFrame via `transform()` — it has no learning step (its parameters are already fixed). A **Estimator** has a `fit(df)` method which learns parameters from data and returns a fitted Transformer. In a Pipeline, Estimator stages are fit on training data and then converted to Transformers; at test time the PipelineModel runs only the Transformer `transform()` steps in sequence.

Q: What is the danger of fitting preprocessing (e.g. StandardScaler) on the full dataset before splitting into train and test?

> This causes **data leakage**: the scaler's mean and standard deviation are computed using test-set values, so the model indirectly "sees" the test data during training. This gives an optimistically biased estimate of model performance. The correct approach is to fit the scaler *only on training data*, then apply (transform) it to both train and test. Using Spark ML Pipelines with CrossValidator ensures this is done correctly on each fold.

Q: When should you use One-Hot Encoding instead of Ordinal Encoding?

> Use One-Hot Encoding when the categorical feature has **no natural order** among its values (e.g. country, colour, product category). Ordinal encoding assigns integer codes that imply a numeric ordering; if no such ordering exists, the model may incorrectly learn that one category is "greater" than another. OHE avoids this by creating separate binary features — one per category — with no implied ordering.